Functional Verification of a 5-Stage RV32IM Pipeline Without Hazard Handling

Jonathan Avalos

***Abstract*—This paper describes the first test of a 5-stage pipelined RV32IM processor written in Verilog. The pipeline has no forwarding, stalls or flushes, so a program only runs correctly if it spaces out dependent instructions itself. A 143-instruction test program was written to follow these spacing rules while exercising loads, stores, signed and unsigned branches, loops, a function call, and the multiply and divide unit. A self-checking testbench compares 10 results against known answers and can print a cycle-by-cycle register trace. All 10 results were correct on both the pipelined and the original single-cycle core. The pipeline took 471 cycles against 366 for the single-cycle core, and the 105-cycle difference is fully explained by 51 taken branches and jumps at 2 cycles each, plus 3 cycles of pipeline depth.**

I. Introduction

The processor under development started as a single-cycle RV32IM core: every instruction is fetched, decoded, executed and written back in one clock cycle [**?**]. The next step is to split it into the classic 5-stage pipeline (IF, ID, EX, MEM, WB) [**?**], with a pipeline register between each pair of stages.

The pipeline was built in two parts. This paper covers the first: the datapath and the four pipeline registers, with *no hazard handling*. Forwarding, a load-use stall and a branch flush are left for the second part.

Testing at this point needed a new program. The existing 141-instruction test for the single-cycle core places dependent instructions next to each other, which a pipeline without hazard handling cannot run correctly. The goal was a program that:

1) obeys the pipeline's spacing rules, so any wrong result points to a bug in the pipeline itself;
2) covers memory access, branches, jumps, and the multiply and divide unit; and
3) has known answers that a testbench can check automatically.

II. Background

A. The Pipeline Under Test

The pipelined core keeps the same modules and ports as the single-cycle core. IF fetches the instruction at `pc`. ID decodes it and reads `rs1` and `rs2`. EX runs the ALU and the multiply and divide unit, computes branch and jump targets, and decides whether to branch. MEM performs the load or store, and WB writes the result to `rd`. The register file is read in ID and written in WB, so its write port is driven from the MEM/WB pipeline register rather than from the instruction being decoded.

B. Spacing Rules

Without hazard handling, two situations break a program.

Data hazards. A result is written into the register file at the clock edge at the end of WB, and a later instruction reads its operands in ID. The register file has no write-through, so the read must happen in the cycle after the write. Table I shows that this requires 3 instructions in between.

TABLE I
WRITE FOLLOWED BY A READ OF THE SAME REGISTER

cycle	1	2	3	4	5	6
`addi x1, x0, 5`	IF	ID	EX	MEM	**WB**	
`nop`		IF	ID	EX	MEM	WB
`nop`			IF	ID	EX	MEM
`nop`				IF	ID	EX
reads `x1`					IF	**ID**

Control hazards. Branches and jumps are resolved in EX. By then the 2 instructions after the branch are already in the pipeline, and with no flush they run whether the branch is taken or not (Table II).

TABLE II
TAKEN BRANCH RESOLVED IN EX

cycle	1	2	3	4
`bne`	IF	ID	**EX**	MEM
next		IF	ID	EX
next + 1			IF	ID
target				IF

This gives two rules for the test program: at least **3 instructions** between writing a register and reading it, and **2 `nop`s** after every branch and jump. The 3 instructions in the first rule may be any instructions that do not use the register.

III. Test Design

A. Memory Layout

The program occupies `0x000–0x23B`, the 10 result words occupy `0x400–0x427`, and a 6-word input array sits at `0x600`. The build script extracts only the `.text` section, so the array is placed at the end of `.text` with an `.org 0x600` directive. The assembler pads the gap with zeros, and `$readmemh` loads the array together with the program. As a side effect, the result words start at zero instead of undefined.

TABLE III
PROGRAM SECTIONS, EXPECTED RESULTS AND MEASURED RESULTS

#	result	register	start value	key instruction	expected	measured	
0	sum	x12	0	add	00000021 (33)	00000021	pass
1	max, signed	x13	80000000	bge x13, x5	00000019 (25)	00000019	pass
2	min, signed	x14	7FFFFFFF	bge x5, x14	FFFFFFF8 (-8)	FFFFFFF8	pass
3	negative count	x15	0	bge x5, x0	00000002	00000002	pass
4	max, unsigned	x16	0	bgeu x16, x5	FFFFFFFD	FFFFFFFD	pass
5	product, skipping 0	x17	1	beq x5, x0, mul	0000C4E0 (50400)	0000C4E0	pass
6	sum / 6	x20		div	00000005	00000005	pass
7	sum mod 6	x21		rem	00000003	00000003	pass
8	gcd(48, 18)	x10		jal, jalr, rem	00000006	00000006	pass
9	completion marker	x7		lui, addi	D0E0000D	D0E0000D	pass

Every expected result is non-zero, so a missing store is still detected.

B. Test Program

The array is $\{7, -3, 12, 0, 25, -8\}$. The program has 9 sections, six of which loop once over the array. Each section stores one result at `0x400 + 4n` (Table III). Start values were chosen so the first element always replaces them: the most negative signed number for max, the largest signed number for min, and zero for unsigned max. Results 2 and 4 separate signed from unsigned comparison. As a signed number -8 is the smallest element, but as an unsigned number -3 (`FFFFFFFD`) is the largest. A branch unit that treated `bgeu` like `bge` would produce `00000019` for result 4.

Every loop section has the same shape, shown for the signed maximum in Fig. 1. Each update is skipped by a branch that tests the *opposite* of the condition, so "if $x >$ *max*, update" becomes "if *max* $\geq x$, skip." Gaps were filled with useful instructions where possible. The counter and pointer updates come directly after the `lw`; they do not depend on the loaded value, so they cover most of the load's wait and only one `nop` is needed instead of three.

```
# max (signed)
    addi x18, x0, 6 # counter
    lui x13, 0x80000 # max
    nop # wait for x9
max_loop: # if x > max, max = x
    lw x5, 0(x9)
    addi x18, x18, -1 # counter--
    addi x9, x9, 4 # array pointer++
    nop # wait for lw
    bge x13, x5, max_skip # if max >= x, skip
    nop
    nop # wait for branch target
    add x13, x0, x5 # max = x
max_skip:
    bne x18, x0, max_loop
    nop
    nop # wait for branch target
    sw x13, 4(x8) # store max in box 1
    addi x9, x0, ARRAY # array pointer reset
```

Fig. 1. The signed maximum section. Every write is at least 3 instructions before its read, and every branch is followed by 2 `nop`s.

The greatest common divisor is computed by a function called with `jal x1, gcd`, taking arguments in `x10` and `x11` and returning in `x10` (Fig. 2). It is placed after the final infinite loop so the program cannot fall into it. For 48 and 18 it loops 3 times, with remainders 12, 6 and 0.

```
gcd:
    beq x11, x0, gcd_done # if b == 0, done
    nop
    nop # wait
    rem x5, x10, x11 # t = a % b
    addi x10, x11, 0 # a = b
    nop
    nop # wait for rem
    addi x11, x5, 0 # b = t
    j gcd
    nop
    nop # wait
gcd_done:
    jalr x0, 0(x1) # return
    nop
    nop # wait
```

Fig. 2. The gcd function, called with `jal x1, gcd`.

C. Testbench

The testbench connects the pipelined core to a shared memory and runs in Icarus Verilog [**?**]; the program is assembled with LLVM [**?**].

Completion. The testbench does not assume a cycle count. It waits for the completion marker in result 9, counting cycles, and gives up after 2000 cycles. A program stuck in a loop is therefore reported as a failure instead of hanging the simulation.

Checks. Each result is compared with the `!==` operator, so an undefined value counts as a failure. On the pipeline the PC does not settle on one address at the end: IF keeps fetching the 2 words after the final `j end` before the jump in EX takes effect. The final PC check therefore accepts any address from `1F8` to `200`.

Register trace. When compiled with `-DWATCH`, a `$monitor` prints a line whenever any watched register changes. The register file is written only at the end of WB, so each printed change marks one instruction leaving the pipeline.

IV. Results

A. Functional Results

All 10 results and the final PC check passed on the pipelined core (Table III). The single-cycle core produced the same 10 values from the same program.

B. Register Trace

Table IV shows the first pass of the sum loop. One clock period is 10 000 ps. Writes appear one per cycle and in program order, and every gap matches instructions that do not write a watched register. The 4-cycle gap before the second load is `bne`, the 2 `nop`s after it, and the `lw`. The `nop`s ran even though the branch was taken, as Table II predicts.

TABLE IV
START OF THE REGISTER TRACE

time (ps)	change	instruction leaving WB	Δ
65 000	`x9 = 600`	`addi x9, x0, ARRAY`	
75 000	`x18 = 6`	`addi x18, x0, 6`	1
105 000	`x5 = 7`	`lw x5, 0(x9)`	3
115 000	`x18 = 5`	`addi x18, x18, -1`	1
125 000	`x9 = 604`	`addi x9, x9, 4`	1
145 000	`x12 = 7`	`add x12, x12, x5`	2
185 000	`x5 = FFFFFFFD`	next `lw`	4

The rest of the trace behaved as expected. The min start value appeared as `80000000` and then `7FFFFFFF` 4 cycles later, matching the `lui`–3 instructions–`addi` spacing. The signed max changed only on 7, 12 and 25. The product passed through -21, -252 and -6300 before reaching 50 400, and did not change on the zero element. During the gcd call the temporary register took the values 12, 6 and 0.

C. Cycle Count

The pipelined core reached completion in 471 cycles and the single-cycle core in 366, a difference of 105 cycles. Almost all of it comes from taken branches and jumps. On the single-cycle core a taken branch goes straight to its target, so the 2 `nop`s after it never run; on the pipeline they are already fetched and do run, costing 2 cycles each. The program has 51 taken branches and jumps: 30 loop branches, 15 skip branches, and 6 in the gcd call, for a total of 102 cycles. The remaining 3 cycles are pipeline depth: the completion marker is stored in MEM, 3 stages after the instruction is fetched, while the single-cycle core stores it in the same cycle. Together these account for the full difference.

V. Discussion

A. What the Test Shows

Every result was correct and the trace showed instructions leaving the pipeline one per cycle, in order. This indicates that each pipeline register carries its signals correctly, that the register file is written from WB, and that the branch and jump logic in EX returns the correct target to IF. Because the cycle count was accounted for exactly, a hidden extra stall or a lost instruction would also have been detected.

At this stage the pipeline is slower than the single-cycle core. It still completes at most one instruction per cycle, but it loses 2 cycles on every taken branch, and the program carries extra `nop`s. The benefit of pipelining, a shorter clock period, will only appear once the design is synthesized for hardware.

B. Limitations

The program was written to *avoid* hazards, so it says nothing about the pipeline's behavior when they occur. It also does not cover byte and halfword memory access, `auipc`, shifts, or the remaining multiply and divide variants.

C. Errors Found During Development

Writing the program by hand to these rules was error-prone (Table V). Several of the errors assembled without warnings and were only caught because the testbench checked known answers. The first is a direct example of a data hazard: one missing instruction in a gap caused a stale register read, and a loop meant to run 6 times would have run about 4 billion times.

TABLE V
ERRORS FOUND WHILE WRITING THE TEST PROGRAM

symptom	cause → fix
program never finished	missing `nop` before the first loop; the counter read a stale `x18` of 0 → one more instruction in the gap
min = `7FFFFFFF`	skip branch was `blt` → `bge`, the opposite of the condition
max = last element	1 `nop` after the skip branch → 2 `nop`s after every branch
loop past the array	skip branch targeted the loop top → skip label on the loop branch
max start `08000000`	`lui` given 4 hex digits → `lui x13, 0x80000`
assembler error	`addi` with a 32-bit immediate → `lui`
sum lost before divide	max reused the sum register → one register per result
all results undefined	wrong hex file name in testbench → match the name

VI. Conclusion

A 5-stage RV32IM pipeline without hazard handling was verified with a purpose-built test program and a self-checking testbench. All 10 results were correct, the register trace matched the program instruction by instruction, and the 105-cycle difference from the single-cycle core was fully explained by taken branches and pipeline depth.

Future work is to add forwarding from the EX/MEM and MEM/WB registers, a load-use stall, and a flush on taken branches and jumps. The original test program should then pass on the pipeline unchanged, and this program should pass in fewer cycles once its `nop`s are removed.

APPENDIX A
TEST PROGRAM SOURCE

The complete `test_asm_nohazards.S`, as run for the results above.

```
# To test the core_pipe.v module
# this programs will run with no hazrd
     handling
# since it is not implemented yet.
# 3 nops between writes and reads to the
     same register
# 2 nopes between branches and jumps
# array of 6 words at 0x00000600: 7, -3,
     12, 0, 25, -8
# answers in boxes at 0x00000400 + 4x
#
# [0] sum               00000021
# [1] max (signed)      00000019
# [2] min (signed)      FFFFFFF8
# [3] negatives         00000002
# [4] max (unsigned)    FFFFFFFD
# [5] product (no 0s)   0000C4E0
# [6] sum / 6           00000005
# [7] sum % 6           00000003
# [8] gcd(48, 18)       00000006
# [9] DONE              D0E0000D
    .equ ARRAY, 0x600
    .text
    .globl _start
_start:
    addi x8, x0, 0x400 # x8 is box 0
    addi x9, x0, ARRAY # array pointer
# sum
    addi x18, x0, 6 # counter
    addi x12, x0, 0 # sum
    nop
sum_loop:
    lw x5, 0(x9)
    addi x18, x18, -1 # counter--
    addi x9, x9, 4 # array pointer++
    nop # wait for lw
    add x12, x12, x5 # sum += array[i]
    bne x18, x0, sum_loop
    nop
    nop # wait for branch target
    sw x12, 0(x8) # store sum in box 0
    addi x9, x0, ARRAY # array pointer
     reset
# max (signed)
    addi x18, x0, 6 # counter
    lui x13, 0x80000 # max
    nop # wait for x9
max_loop: # if x > max, max = x
    lw x5, 0(x9)
    addi x18, x18, -1 # counter--
    addi x9, x9, 4 # array pointer++
    nop # wait for lw
    bge x13, x5, max_skip # if max >= x,
     skip
    nop
    nop # wait for branch target
    add x13, x0, x5 # max = x
max_skip:
    bne x18, x0, max_loop
    nop
    nop # wait for branch target
    sw x13, 4(x8) # store max in box 1
    addi x9, x0, ARRAY # array pointer
     reset

# min (signed)
    lui x14, 0x80000 # min
    addi x18, x0, 6 # counter
    nop # wait for x9
    nop
    addi x14, x14, -1 # min = 0x7FFFFFFF
min_loop: # if x < min, min = x
    lw x5, 0(x9)
    addi x18, x18, -1 # counter--
    addi x9, x9, 4 # array pointer++
    nop # wait for lw
    bge x5, x14, min_skip # if x >= min,
     skip
    nop
    nop # wait for branch target
    add x14, x0, x5 # min = x
min_skip:
    bne x18, x0, min_loop
    nop
    nop # wait for branch target
    sw x14, 8(x8) # store min in box 1
    addi x9, x0, ARRAY # array pointer
     reset

# negatives
# neg = 0, loop, if x < 0, neg++, counter--
    addi x15, x0, 0 # neg
    addi x18, x0, 6 # counter
    nop
    nop
neg_loop:
    lw x5, 0(x9)
    addi x18, x18, -1 # counter--
    addi x9, x9, 4 # array pointer++
    nop # wait for lw
    bge x5, x0, neg_skip # if x >= 0, skip
    nop
    nop # wait
    addi x15, x15, 1 # neg++
neg_skip:
    bne x18, x0, neg_loop
    nop
    nop # wait
    sw x15, 12(x8) # store neg in box 3
    addi x9, x0, ARRAY # array pointer
     reset

# max (unsigned)
# umax = 0, loop, if x > umax (unsigned),
     umax = x
    addi x18, x0, 6 # counter
    addi x16, x0, 0 # umax
    nop
    nop
umax_loop:
    lw x5, 0(x9)
    addi x18, x18, -1 # counter--
    addi x9, x9, 4 # array pointer++
    nop # wait for lw
    bgeu x16, x5, umax_skip # if umax >= x,
     skip
    nop
    nop # wait
    add x16, x0, x5 # umax = x
umax_skip:
    bne x18, x0, umax_loop
    nop
    nop # wait
    sw x16, 16(x8) # store umax in box 4
    addi x9, x0, ARRAY # array pointer
     reset

# product
# prod = 1, loop, if x != 0, prod *= x
    addi x18, x0, 6 # counter
    addi x17, x0, 1 # prod
    nop
    nop
prod_loop:
    lw x5, 0(x9)
    addi x18, x18, -1 # counter--
    addi x9, x9, 4 # array pointer++
    nop # wait for lw
    beq x5, x0, prod_skip # if x == 0, skip
    nop
    nop # wait
    mul x17, x17, x5 # prod *= x
prod_skip:
    bne x18, x0, prod_loop
    nop
    nop # wait
    sw x17, 20(x8) # store prod in box 5

# divide
# sum is still in x12
    addi x19, x0, 6 # six
    nop
    nop
    nop # wait for x19
    div x20, x12, x19 # sum / 6
    rem x21, x12, x19 # sum % 6
    nop
    nop # wait for div, rem
    sw x20, 24(x8) # store sum / 6 in box 6
    sw x21, 28(x8) # store sum % 6 in box 7

# gcd
# a in x10, b in x11, answer in x10
    addi x10, x0, 48 # a
    addi x11, x0, 18 # b
    nop
    nop
    jal x1, gcd # call gcd
    nop
    nop # wait
    sw x10, 32(x8) # store gcd in box 8

# done
    lui x7, 0xD0E00
    nop
    nop
    nop # wait for lui
    addi x7, x7, 0x0D # x7 = D0E0000D
    nop
    nop
    nop # wait for x7
    sw x7, 36(x8) # store DONE in box 9
end:
    j end
    nop
    nop

# gcd function
# while b != 0: t = a % b, a = b, b = t
gcd:
    beq x11, x0, gcd_done # if b == 0, done
    nop
    nop # wait
    rem x5, x10, x11 # t = a % b
    addi x10, x11, 0 # a = b
    nop
    nop # wait for rem
    addi x11, x5, 0 # b = t
    j gcd
    nop
    nop # wait
gcd_done:
    jalr x0, 0(x1) # return
    nop
    nop # wait

    .org 0x600
array:
    .word 7, -3, 12, 0, 25, -8
```